have a dedicated Set class, which we'll cover in Chapter 27, Library Reference: Enumerators and Containers, on page 529.)

For example, push and pop add and remove elements from the end of an array, so you can use the array as a stack:

```
stack = []
stack.push "red"
stack.push "green"
stack.push "blue"
stack     # => ["red", "green", "blue"]

stack.pop # => "blue"
stack.pop # => "green"
stack.pop # => "red"
stack     # => []
```

Similarly, unshift and shift add and remove elements from the beginning of an array. Combine shift and push, and you have a first-in-first-out (FIFO) queue.

```
queue = []
queue.push "red"
queue.push "green"
queue.shift # => "red"
queue.shift # => "green"
```

The first and last methods return (but don't remove) the *n* entries at the head or end of an array. If you don't pass an argument, the default number is one.

```
array = [1, 2, 3, 4, 5, 6, 7]
array.first    # => 1
array.first(4) # => [1, 2, 3, 4]
array.last     # => 7
array.last(4)  # => [4, 5, 6, 7]
```

We'll look at more of the methods of array later on in Chapter 5, More About Methods, on page 85.

Hashes

Hashes (sometimes known as *associative arrays*, *maps*, or *dictionaries*) are similar to arrays in that they are indexed collections of object references. However, while you index arrays with integers, you index a hash with objects of any type, most often symbols and strings but also regular expressions or anything else in Ruby. When you store a value in a hash, you actually supply two objects—the index, which is called the *key*, and the *value*, or entry to be stored with that key. You can subsequently retrieve the entry by indexing the hash with the same key value that you used to store it.

Why Are They Called Hashes?



The data structure that Ruby calls a *Hash*—where an arbitrary key is an index to an arbitrary value—has different names in different programming languages. The most generic term is probably *key-value store*. You will also see them called *dictionaries* because the values are looked up based on the keys, or *map* because